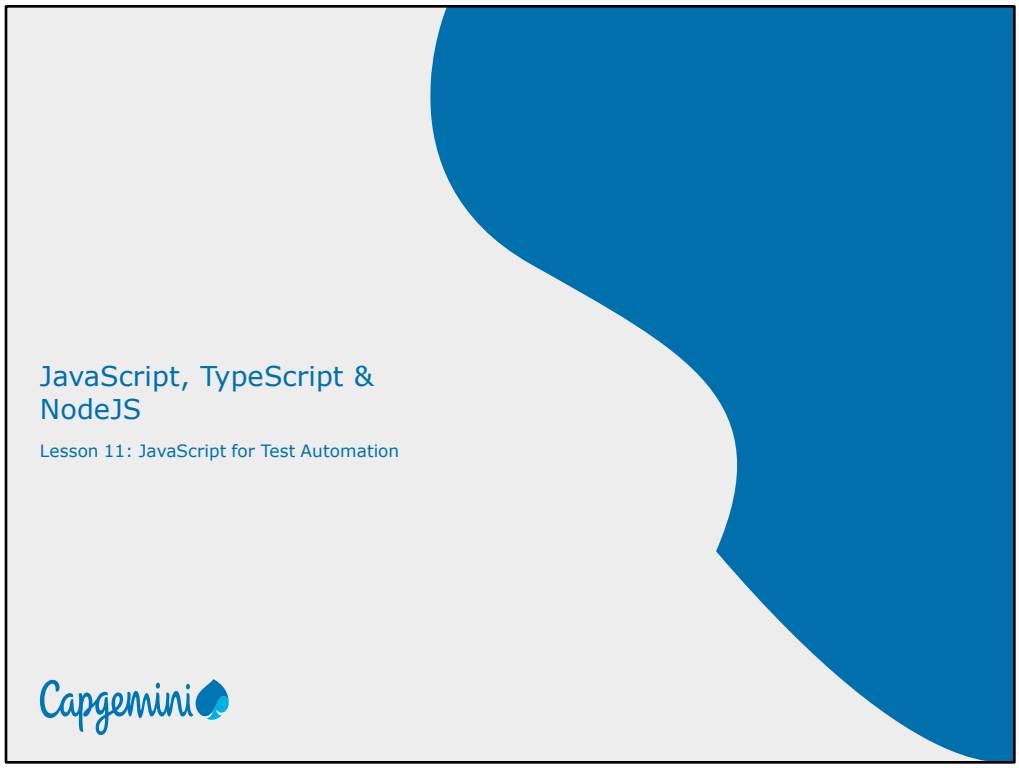
JavaScript, TypeScript &
NodeJS
Lesson 11: JavaScript for Test Automation
Capgemini

Lesson Objectives

- Introduction to Testing Frameworks (Mocha, Jest, Jasmine)
- Setting up and Running Tests with Mocha
- Writing Unit Tests in JavaScript
- Testing Asynchronous Code
- Mocking and Stubbing
- Best Practices for Testing

11.1: JavaScript for Test Automation

Introduction to Testing Frameworks

- **Why frameworks?** They provide a structured way to test code automatically.
- **Mocha**: Flexible testing framework, often paired with assertion libraries like Chai.
- **Jest**: Popular for JavaScript/React projects, comes with built-in assertions, mocks.
- **Jasmine**: Older but widely used, has everything built-in (no external libraries needed).
- *Think of frameworks as examiners checking your code's correctness.*

11.2: JavaScript for Test Automation

Setting up and Running Tests with Mocha

- Mocha is a popular JavaScript test framework, often used for both front-end and back-end testing.
- It provides a flexible and feature-rich environment for writing and running tests, supporting both synchronous and asynchronous code.
- Mocha is known for its simple interface, detailed test reports, and ability to integrate with other testing tools like Chai for assertions.
- Setting up Mocha:
  - Install Mocha: npm install --save-dev mocha
  - Write test files inside test/ folder.

```
const assert = require('assert');
describe('Array', function() {
  it('should return -1 when value is not present',
function() {
    assert.equal([1,2,3].indexOf(4), -1);
  });
});
```

11.3: JavaScript for Test Automation

Writing Unit Tests in JavaScript

- **Unit Test**: Tests a small, isolated piece of code.

```
function add(a, b) { return a + b; }
module.exports = add;
// test
const assert = require('assert');
const add = require('./add');
assert.equal(add(2, 3), 5);
```

11.4: JavaScript for Test Automation

Testing Asynchronous Code

- Many JS functions (API calls, file reads) run asynchronously.
- Mocha handles async using done callback or async/await.

```
it('should fetch data from API', async () => {
  let data = await fetchData();
  assert.equal(data.status, 200);
});
```

11.5: JavaScript for Test Automation

Mocking and Stubbing

- Mocking:
  - A mock is a simulated object or function.
  - It behaves like the real one but does not perform actual work.
  - Used to verify interactions (e.g., “Was this function called?”).
- Stubbing:
  - A stub replaces a real function with controlled behavior.
  - It doesn’t just pretend; it returns fixed values for testing.
  - Used to isolate code and avoid external dependencies.
- Example (API Testing)
- Instead of calling the real API (which may be slow or unavailable),we create a stubbed response.

11.5: JavaScript for Test Automation

Mocking and Stubbing

```
const stubbedAPI = () =>
Promise.resolve({ status: 200, data: "OK"
});
```

- This way, tests run faster and are independent of the real server.
- Key Difference:Mocks → Check how code interacts with dependencies.Stubs → Provide fake data to control test behavior.

11.6: JavaScript for Test Automation

Best Practices for Testing

- Write small, independent tests.
- Use meaningful names (shouldReturnSumWhenValidNumbers).
- Test both success & failure cases.
- Keep tests fast and repeatable.

Review Question

Question 1: Fill in the blanks.
_____ framework provide built-in mocking and assetion

Question 2: What is the purpose of mocking in testing?

Question 3: State True or False:
In Mocha, npx mocha is used to run tests after writing them?

Question 1 – Jest
Question 2 – To simulate behavior of external dependencies
Question 3 – True

Summary

In this lesson, you have learnt

- Introduction to Testing Frameworks (Mocha, Jest, Jasmine)
- Setting up and Running Tests with Mocha
- Writing Unit Tests in JavaScript
- Testing Asynchronous Code
- Mocking and Stubbing
- Best Practices for Testing

Add the notes here.